



Lesson 3

Composite Data Types



What is in this lesson?

Composite data types

Enumerations

Dictionaries and Sets in Python

Primitive Data Types (recap)

Primitive data types are the basic, built-in data types in programming languages. They represent single values and are immutable.

- ▶ `int`: integer numbers
- ▶ `float`: real numbers
- ▶ `bool`: carry one of two values `True` or `False`
- ▶ `char`: one single character

Primitive data types are essential for creating more complex data types called **composite data types**.

Composite data types

Composite Data Types

- ▶ **Composite data types** are complex, user-defined structures built from primitives data types.
- ▶ They are either included in the programming language or created by programmers.
- ▶ array
- ▶ tuple
- ▶ dict
- ▶ set
- ▶ Classes and Objects
- ▶ Abstract data types representing complex entities in a program

Composite Data Types: An Example

Let's take student data as an example.

Composite Data Types: An Example

Let's take student data as an example.

Field	Data Type	Complex/Basic	Example/format
id	String	Basic	1023450X
<u>first_name</u>	String	Basic	Jenny
<u>last_name</u>	String	Basic	<u>Fortesecue</u>
<u>email_address</u>	String	Basic	<u>jforte@gmail.com</u>
<u>home_address</u>	Address	Complex	Street number, street name, suburb, state, postcode, country.
DOB	Date	Complex	<u>dd/mm/yyyy</u>
<u>phone_number</u>	String	Basic	0414 899 456
<u>enrolled_course</u>	String	Basic	BA-ICT
<u>unit_results</u>	Results	Complex	<u>unit_code</u> , mark, grade, semester, year (multiple)
<u>unit_enrolments</u>	Enrolment	Complex	<u>unit_code</u> , semester, year

C Structures (struct)

- ▶ A **struct** in C is a programmer-created data type that combines different types of variables in a structure variable.
- ▶ The variables are called struct **members**.

```
struct Date {  
    int day;  
    char month[10];  
    int year;  
};
```

```
struct Date d;  
d.day = 5;  
d.year = 2026;  
  
int y = d.year;
```


Python Classes [1, Ch. 10]

A **class** specifies the attributes and methods for a particular type.

```
1 class Date:
2     def __init__(self, day: int, month: str, year: int):
3         self.day = day
4         self.month = month
5         self.year = year
```

- ▶ The `self` parameter is required in every method of a class. It references the specific object that the method is supposed to operate on.
- ▶ The `__init__` method is known as an initialiser method since it initialises the object's data attributes.

Using Composite Data Types

- ▶ In Python, **class** is considered as a blueprint of the custom data type that we define.
- ▶ To use it as a variable for storing custom data, we need to create an **instance** of the class.

```
1 d = Date(5, "January", 2026)
2 d.day = 12
```

Datetime Composite Data Type in Python

Python provides a built-in datetime module for handling dates and times.

```
1 from datetime import date
2
3 # Create a date object for January 05, 2026
4 meeting_date = date(year=2026, month=1, day=5)
5
6 print(meeting_date)
7 # Output: 2026-01-05
```

Nested Composite Data Types

A composite data type can have fields/ members/ attributes that are also composite data types.

```
1 class Date:
2     def __init__(self, day: int, month: str, year: int):
3         self.day = day
4         self.month = month
5         self.year = year
6
7 class Booking:
8     def __init__(self, room: str, date: Date, time: str):
9         self.room = room
10        self.date = date
11        self.time = time
```

Enumerations

C Enumerations (enum)

- ▶ Enumeration is a composite data type, holds constant values.
- ▶ Enumeration assigns meaningful names to a set of integers.

```
enum Month {  
    JANUARY = 1, FEBRUARY, MARCH, APRIL, MAY, JUNE,  
    JULY, AUGUST, SEPTEMBER, OCTOBER, NOVEMBER, DECEMBER  
};
```

```
struct Date {
    int day;
    enum Month month;
    int year;
};

int main() {
    enum Month m = JANUARY;
    printf("Month number: %d\n", m);
    return 0;
}
```

```
enum Month {  
    JANUARY = 1, FEBRUARY, MARCH, APRIL, MAY, JUNE,  
    JULY, AUGUST, SEPTEMBER, OCTOBER, NOVEMBER, DECEMBER  
};
```

- ▶ Enumerations are by default numbered incrementally in ascending order;
- ▶ but you can break this by explicitly defining the number directly.
- ▶ enum values start at 0 by default.
- ▶ In C, an enum is internally represented as an int.

Why use constants and enumerations?

- ▶ To give meaning to numbers, improve readability
- ▶ Prevent errors
- ▶ Easier to maintain
- ▶ Logical grouping of related values

Dictionaries and Sets in Python

Dictionaries [1, Ch. 9]

Concept

A **dictionary** is an object that stores a collection of data.

- ▶ Each element in a dictionary has a pair of: **key** and **value**.
- ▶ A **key** is used to locate a specific **value**.
- ▶ You can create a dictionary by enclosing the elements inside a set of curly braces (`{}`).

```
1 phonebook = {  
2     'Chris': '555-1111',  
3     'Katie': '555-2222',  
4     'Joanne': '555-3333'  
5 }
```

Retrieving, Adding, and Removing Dictionary Elements [1, Ch. 9]

- ▶ Retrieve a value:

```
1 <dictionary_name>[<key>]
```

- ▶ Add a new key-value pair or assign a value to an existing key:

```
1 <dictionary_name>[<key>] = <value>
```

- ▶ Remove an existing key-value pair:

```
1 del <dictionary_name>[<key>]
```

The in and not in Operators [1, Ch. 9]

- ▶ Accessing a nonexistent **key** in a dictionary raises a `KeyError`.
- ▶ Use the `in` operator to check if a **key** exists before accessing it.
- ▶ Use `not in` to check if a **key** does not exist.

```
1 if 'Chris' in phonebook:  
2     print(phonebook['Chris'])  
3  
4 if 'Tom' not in phonebook:  
5     print('Tom is not found.')
```

Iterate over a Dictionary [1, Ch. 9]

You can use the for loop to iterate over all the keys in a dictionary.

```
1 for <key> in <dictionary>:  
2     <statement>  
3     <statement>
```

- ▶ This loop iterates once for each element in the dictionary.
- ▶ Each time the loop iterates, the variable <key> is assigned a **key**.

Sets [1, Ch. 9]

Concept

A **set** is an object that stores a collection of unique values in the same way as mathematical sets.

- ▶ All the elements in a **set** must be unique.
- ▶ **Sets** are unordered.
- ▶ The elements that are stored in a **set** can be of different data types.

```
1 # Create an empty set
2 myset = set()
3 # Create a set with some elements
4 myset = set(['a', 'b', 'c'])
```

Adding and Removing Elements [1, Ch. 9]

- ▶ Add a new element:

```
1 <Set>.add(<value>)
```

- ▶ Remove an existing element:

```
1 <Set>.remove(<value>)
```

- ▶ Discard an element:

```
1 <Set>.discard(<value>)
```

The only difference between `remove` and `discard` is that when the specified item is not found in the set, the `remove` method raises a `KeyError`, but the `discard` method does not.

Iterate over a Set [1, Ch. 9]

You can use the for loop to iterate over all the elements in a set.

```
1 for <element> in <Set>:  
2     <statement>  
3     <statement>
```

Similarly, you can use the `in` and `not in` operators to check if an element exists in a set.

Set Operations [1, Ch. 9]

Given two sets `set1` and `set2`, in Python:

► `set1 ∪ set2`

```
1 set1.union(set2)
2 set1 | set2
```

► `set1 ∩ set2`

```
1 set1.intersection(set2)
2 set1 & set2
```

► `set1 \ set2`

```
1 set1.difference(set2)
2 set1 - set2
```

► `set1 ⊆ set2`

```
1 set1.issubset(set2)
```

► `set1 ⊇ set2`

```
1 set1.issuperset(set2)
```

Recap

In this lecture, we've learned:

- ▶ What are composite data types, and what are some composite data types in C and Python?
- ▶ Nested composite data types
- ▶ Enumerations in C
- ▶ Dictionaries and Sets in Python

References



Gaddis, Tony *Starting out with Python*. Pearson Education Limited. Sixth Edition, 2024.

Any Question?